

华中科技大学

课程实验报告

课程名称：计算机系统基础

专业班级 2401

学号 U202490042

姓名 马俊豪

指导教师 张宇

报告日期 2026 年 5 月 8 日

计算机科学与技术学院

目 录

实验 1: 数据表示和等效运算	1
实验 2: 汇编和 C 的混合编程及优化	13
实验 3: 机器级语言理解 (二进制炸弹拆除)	21
实验 4: 缓冲区溢出攻击	30
实验 5: ELF 文件与程序链接	35
6 实验总结	43
参考文献	44

实验 1：数据表示和等效运算

1.1 实验概述

1. 熟练掌握程序开发平台 (VS2019/Vs2022/Vs2023) 的基本用法，包括程序的编译、链接和调试；
2. 熟悉数据的表示形式；
3. 熟悉地址的计算方法、地址的内存转换；
4. 用常见的按位操作（移位、按位与/或/非/异或）等实现运算表达式的等效运算。

1.2 实验内容

任务 1 数据存放的压缩与解压编程

定义了结构 `student`，以及结构数组变量 `old_s[N]`, `new_s[N]`; ($N=5$)

```
1 struct student {  
2     char   name[8];  
3     short  age;  
4     float  score;  
5     char   remark[200]; // 备注信息  
6 };
```

编写程序，输入 N 个学生的信息到结构数组 `old_s` 中。将 `old_s[N]` 中的所有信息依次紧凑 (压缩) 存放到一个字符数组 `message` 中，然后从 `message` 解压缩到结构数组 `new_s[N]` 中。打印压缩前 (`old_s`)、解压后 (`new_s`) 的结果，以及压缩前、压缩后存放数据的长度。

要求：

1. 输入的第 0 个人姓名 (`name`) 为自己的名字，分数为学号的最后两位；
2. 编写指定接口的函数完成数据压缩

压缩函数有两个：

```
1 int pack_student_bytebybyte(student* s, int sno, char *buf);  
2 int pack_student_whole(student* s, int sno, char *buf);
```

s 为待压缩数组的起始地址；sno 为压缩人数；buf 为压缩存储区的首地址；两个函数的返回均是调用函数压缩后的字节数。pack_student_bytebybyte 要求一个字节一个字节的向 buf 中写数据；pack_student_whole 要求对 short、float 字段都只能用一条语句整体写入，用 strcpy 实现串的写入。

3. 使用指定方式调用压缩函数

old_s 数组的前 N1 (N1=2) 个记录压缩调用 pack_student_bytebybyte 完成；后 N2 (N2=3) 个记录压缩调用 pack_student_whole，两种压缩函数都只调用 1 次。

4. 使用指定的函数完成数据的解压

解压函数的格式：

```
1 int restore_student(char *buf, int len, student* s);
```

buf 为压缩区域存储区的首地址；len 为 buf 中存放数据的长度；s 为存放解压数据的结构数组的起始地址；返回解压的人数。解压时不允许使用函数接口之外的信息（即不允许定义其他全局变量）

5. 仿照调试时看到的内存数据，以十六进制的形式，输出 message 的前 20 个字节的內容，并与调试时在内存窗口观察到的 message 的前 20 个字节比较是否一致。
6. 对于第 0 个学生的 score，根据浮点数的编码规则指出其个部分的编码，并与观察到的内存表示比较，验证是否一致。
7. 指出结构数组中个元素的存放规律，指出字符串数组、short 类型的数、float 型的数的存放规律。

任务 2 编写位运算程序

按照要求完成给定的功能，并自动判断程序的运行结果是否正确。（从逻辑电路与门、或门、非门等等角度，实现 CPU 的常见功能。所谓自动判断，即用简单的方式实现指定功能，并判断两个函数的输出是否相同。）

1. int absVal(int x); 返回 x 的绝对值

仅使用!、~、&、^、|、+、«、»，运算次数不超过 10 次

判断函数: `int absVal_standard(int x) { return (x < 0) ? -x : x; }`

2. `int negate(int x)`; 不使用负号, 实现 $-x$
判断函数: `int negate_standard(int x) { return -x; }`
3. `int bitAnd(int x, int y)`; 仅使用 $\sim$ 和 $|$, 实现 $\&$
判断函数: `int bitAnd_standard(int x, int y) { return x & y; }`
4. `int bitOr(int x, int y)`; 仅使用 $\sim$ 和 $\&$, 实现 $|$
5. `int bitXor(int x, int y)`; 仅使用 $\sim$ 和 $\&$, 实现 $\wedge$
6. `int isTmax(int x)`; 判断 x 是否为最大的正整数 ($7FFFFFFF$),
只能使用 $!$ 、 $\sim$ 、 $\&$ 、 $\wedge$ 、 $|$ 、 $+$
7. `int bitCount(int x)`; 统计 x 的二进制表示中 1 的个数
只能使用, $! \sim \& ^ | + \ll \gg$, 运算次数不超过 40 次
8. `int bitMask(int highbit, int lowbit)`; 产生从 `lowbit` 到 `highbit` 全为 1, 其他位为 0 的数。例如 `bitMask(5,3) = 0x38`; 要求只使用 $! \sim \& ^ | + \ll \gg$; 运算次数不超过 16 次。
9. `int addOK(int x, int y)`; 当 $x+y$ 会产生溢出时返回 1, 否则返回 0
仅使用 $!$ 、 $\sim$ 、 $\&$ 、 $\wedge$ 、 $|$ 、 $+$ 、 $\ll$ 、 $\gg$, 运算次数不超过 20 次
10. `int byteSwap(int x, int n, int m)`; 将 x 的第 n 个字节与第 m 个字节交换, 返回交换后的结果。 n 、 m 的取值在 0~3 之间。
例: `byteSwap(0x12345678, 1, 3) = 0x56341278`
`byteSwap(0xDEADBEEF, 0, 2) = 0xDEEFBEAD`
仅使用 $!$ 、 $\sim$ 、 $\&$ 、 $\wedge$ 、 $|$ 、 $+$ 、 $\ll$ 、 $\gg$, 运算次数不超过 25 次
11. `int bang(int x)` 当 $x=0$ 时, 返回 1; 其他情况返回 0。实现逻辑非 (!)
例: `bang(3) = 0; bang(0) = 1;`
仅使用 $\sim$ 、 $\&$ 、 $\wedge$ 、 $|$ 、 $+$ 、 $\ll$ 、 $\gg$, 运算次数不超过 12 次
提示: 只有当 $x=0$ 时, x 与 $-x$ 的最高二进制位会同时为 0。
12. `int bitParity(int x)` 当 x 有奇数个二进制位 0, 返回 1; 否则返回 0
例: `bitParity(5) = 0; bitParity(7) = 1;`
仅使用 $!$ 、 $\sim$ 、 $\&$ 、 $\wedge$ 、 $|$ 、 $+$ 、 $\ll$ 、 $\gg$, 运算次数不超过 20 次。
提示: 只有当 x 的高字与低字的对应位 (第 0 位对应第 16 位, 第 1 位对应第 17 位, 依次类推) 同时为 1, 则出现了成对的二进制位 1, 此时, 可以将对应的二进制位置为 0, 不会影响二进制位 1 的个数的奇偶性判断。

1.3 实验设计

本实验的设计围绕“数据在内存中的真实表示”展开，分别从结构体紧凑存储和整数位级运算两个角度实现要求的功能。整体上，任务 1 强调对字节序、类型编码和结构体对齐的理解，任务 2 强调仅利用基本逻辑门等价规则构造常见运算。

1.3.1 任务 1 设计思路

任务 1 采用“按字段顺序串行化，再按相同顺序反串行化”的方案。原始结构体中各字段依次为 name、age、score、remark，其中 name 和 remark 为字符串，age 为 short，score 为 float。若直接存储结构体数组，每个元素会受到字节对齐影响，除有效字段外还会包含填充字节；而压缩后的 message 仅保留实际有用的数据，因此需要将每个学生记录拆解为可连续写入的字节序列。

具体实现时，将前两个学生记录交由 pack_student_bytebybyte 处理。该函数逐字节写入 name 和 remark 的字符内容，并显式补写字符串结束符 '\0'；对于 age 和 score，则先取字段地址，再按底层字节表示逐个写入缓冲区。这样设计的目的是直接观察 short 与 float 在内存中的真实字节序，同时满足“逐字节压缩”的题目要求。

后 3 个学生记录交由 pack_student_whole 处理。该函数对字符串字段使用 strcpy 一次性复制，对 short 和 float 字段使用强制类型转换后整体写入，从而体现“按整体对象写入”的另一种压缩方式。两个压缩函数均返回本次写入的总字节数，主程序将两段长度相加即可得到 message 中有效数据的总长度。

解压部分由 restore_student 完成。该函数维护当前读取位置 cur_len，并严格按照“字符串、short、float、字符串”的顺序恢复一个学生记录。由于压缩时每个字符串都保留了结束符，因此解压时可以借助 strcpy 和 strlen 自动确定字符串长度；而 age 与 score 的读取则直接依据各自字节数推进偏移量。该方法不依赖额外全局信息，只依赖缓冲区首地址和有效长度，满足题目对函数接口的限制。

从数据组织角度看，该设计等价于将结构体数组重新编码为变长记录序列：字符串采用“字符流 + 结束符”存储，数值类型采用机器中的补码或 IEEE 754 单精度浮点编码直接存储。这样既能够比较压缩前后长度差异，也便于结合调试

器内存窗口分析小端机器上的字节排列规律。

1.3.2 任务 2 设计思路

任务 2 的总体策略是将目标函数全部转换为补码运算、逻辑恒等变换和掩码操作，不使用题目限制之外的高级运算。为验证实现正确性，程序对每个自定义函数都编写了一个标准判定版本，并利用随机数据自动对拍；一旦出现不一致，立即输出失败信息并终止，从而保证测试过程具有可重复的判错能力。

在具体方法上，各类函数采用了不同但统一的位级构造思路。`absVal` 和 `negate` 依赖补码表示完成符号翻转与取反加一；`bitAnd`、`bitOr`、`bitXor` 基于德摩根律和逻辑等价式，用受限操作组合出目标逻辑；`isTmax`、`addOK`、`bang` 则通过符号位分析和特殊值判定来识别边界情况。这样的设计体现了“高级判断最终都可以归结为对符号位和进位关系的分析”。

对于较复杂的位操作，程序采用分治和掩码分层统计的思想。`bitCount` 依次构造 `0x55555555`、`0x33333333`、`0x0F0F0F0F` 等掩码，将 32 位整数中的局部计数逐步合并，最终得到全部 1 的个数；`bitMask` 通过构造上下边界掩码并组合，生成指定区间为 1 的结果；`byteSwap` 先提取第 `n`、`m` 个字节，再清空原位置并回填，实现字节级交换；`bitParity` 则不断将高位信息折叠到低位，通过异或累计奇偶性。上述方法避免了循环逐位判断，在操作次数受限的前提下更容易满足题目要求。

因此，实验 1 的设计重点并不只是“写出程序”，而是在实现过程中把结构体布局、字节序、浮点编码、补码表示和布尔代数这些底层原理转化为可执行的代码步骤，并通过自动测试与内存观察相互印证结果的正确性。

1.4 实验过程

1.4.1 环境准备

- 操作系统：Windows 11
- 编译器：g++
- 开发工具：VS Code、C/C++ 调试器、内存查看窗口
- 其他依赖：无额外运行时依赖，实验结果通过控制台输出与调试窗口联合验证

1.4.2 具体实施步骤

1. 在 LAB_1 目录中分别编写 task1.cpp 与 task2.cpp。任务 1 先定义 student 结构体和两个结构数组，任务 2 先给出 12 个受限位运算函数的接口以及对应的标准判定函数，明确每个函数允许使用的运算符范围。
2. 完成任务 1 的压缩与解压逻辑。程序首先固定第 0 个学生信息为 Rocky, 20, 42.0, Great, 再从控制台输入其余 4 个学生数据。随后先输出压缩前的原始结构体数组，再调用 pack_student_bytebybyte 压缩前 2 个学生、调用 pack_student_whole 压缩后 3 个学生，并将两段结果顺序写入字符数组 message。
3. 在完成压缩后，输出 message 前 40 个字节的十六进制内容，并在调试器中于压缩结束位置设置断点，打开内存窗口观察同一地址处的字节序列，比较控制台输出与内存窗口显示是否一致。同时根据 score=42.0 的 IEEE 754 单精度编码分析符号位、阶码和尾数，与实际内存表示进行核对。
4. 调用 restore_student 对 message 中的数据进行反串行化，重新生成结构数组 decompress，再将解压后的 5 条学生记录逐项打印，与压缩前数据进行人工比对，确认姓名、年龄、成绩和备注均保持一致。
5. 完成任务 2 的 12 个位运算函数实现。对每一个目标函数，都编写一个使用普通 C/C++ 运算规则的标准版本，然后在 main 函数中采用随机数测试或边界值测试的方式自动对拍；若发现结果不一致，程序立即输出失败信息并终止，否则输出“测试通过”。
6. 对 isTmax、bang、bitMask、byteSwap 和 bitParity 等容易受边界影响的函数进行重点检查。其中 isTmax 额外测试 0x7FFFFFFF 与 -1，bang 额外测试 0，bitMask 保证生成的随机区间满足 low <= high，从而避免因非法输入影响验证结果。

```
1 # 任务1: 结构体压缩与解压
2 g++ -g task1.cpp -o task1
3 ./task1
4
5 # 任务2: 位运算函数自动测试
6 g++ -g task2.cpp -o task2
7 ./task2
```

Listing 1 实验 1 中使用的编译与运行命令


```
1 int restore_student(char *buf, int len, student *s)
2 {
3     int cur_len = 0, i;
4     for (i = 0; cur_len <= len; i++)
5     {
6         strcpy(s[i].name, buf + cur_len);
7         cur_len = cur_len + strlen(s[i].name) + 1;
8
9         s[i].age = *(short *)(buf + cur_len);
10        cur_len = cur_len + sizeof(short);
11
12        s[i].score = *(float *)(buf + cur_len);
13        cur_len = cur_len + sizeof(float);
14
15        strcpy(s[i].remark, buf + cur_len);
16        cur_len = cur_len + strlen(s[i].remark) + 1;
17    }
18    return i + 1;
19 }
```

Listing 2 任务 1 解压核心代码

1.5 实验结果

1.5.1 结果展示

表 1-1 实验 1 测试结果记录表

测试编号	输入或场景	预期结果	实际结果
Test 1	统计压缩前后字节数	压缩后长度明显减小	1080 B 压缩为 123 B
Test 2	解压后重新输出 5 条记录	各字段与压缩前一致	5 条记录全部一致
Test 3	比对前 40 字节与内存窗口	两处十六进制内容一致	完全一致
Test 4	12 个位运算函数自动对拍	所有测试项均通过	12 项全部通过

```

第0个学生的score 32位浮点数编码:
二进制:0 10000100 010100000000000000000000
十六进制:0x42280000
peter
18
60
peterpeterppter
Jerry
28
100
thefriendoftom
xiaoming
3
10
goodboy
xiaohong
18
80
goodgirl
第1个学生信息如下:
姓名:Rocky
年龄:20
分数:42.00
备注:Great
-----
第2个学生信息如下:
姓名:peter
年龄:18
分数:60.00
备注:peterpeterppter
-----
第3个学生信息如下:
姓名:Jerry
年龄:28
分数:100.00
备注:thefriendoftom
-----
第4个学生信息如下:
姓名:xiaoming
年龄:3
分数:10.00
备注:goodboy
-----
第5个学生信息如下:
姓名:xiaohong
年龄:18
分数:80.00
备注:goodgirl
-----
压缩前存放数据的长度为:1080
压缩后存放数据的长度为:123
message的前40个字节的內容:
52 6F 63 6B 79 00 14 00 00 00 28 42 47 72 65 61 74 00 70 65 74 65 72 00 12 00 00 00 70 42 70 65 74 65 72

```

(a) 任务 1 程序输出第一页

```

message的前40个字节的內容:
52 6F 63 6B 79 00 14 00 00 00 28 42 47 72 65 61 74 00 70 65 74 65 72 00 12 00 00 00 70 42 70 65 74 65 72
第1个学生信息如下:
姓名:Rocky
年龄:20
分数:42.00
备注:Great
-----
第2个学生信息如下:
姓名:peter
年龄:18
分数:60.00
备注:peterpeterppter
-----
第3个学生信息如下:
姓名:Jerry
年龄:28
分数:100.00
备注:thefriendoftom
-----
第4个学生信息如下:
姓名:xiaoming
年龄:3
分数:10.00
备注:goodboy
-----
第5个学生信息如下:
姓名:xiaohong
年龄:18
分数:80.00
备注:goodgirl

```

(b) 任务 1 程序输出第二页

图 1-1 任务 1 控制台运行结果截图

任务 1 的关键输出结果如图1-1所示。程序首先打印第 0 个学生分数 42.0 的 IEEE 754 单精度编码，然后输出压缩前的学生信息、压缩前后字节数以及 message 的前 40 个字节内容。

```

1 第0个学生的score 32位浮点数编码:
2 二进制:0 10000100 010100000000000000000000
3 十六进制:0x42280000
4
5 压缩前存放数据的长度为:1080
6 压缩后存放数据的长度为:123
7 message的前40个字节的內容:
8 52 6F 63 6B 79 00 14 00 00 00 28 42 47 72 65 61 74 00
9 70 65 74 65 72 00 12 00 00 00 70 42 70 65 74 65 72 70 65 74 65 72

```

Listing 3 任务 1 关键运行结果摘录

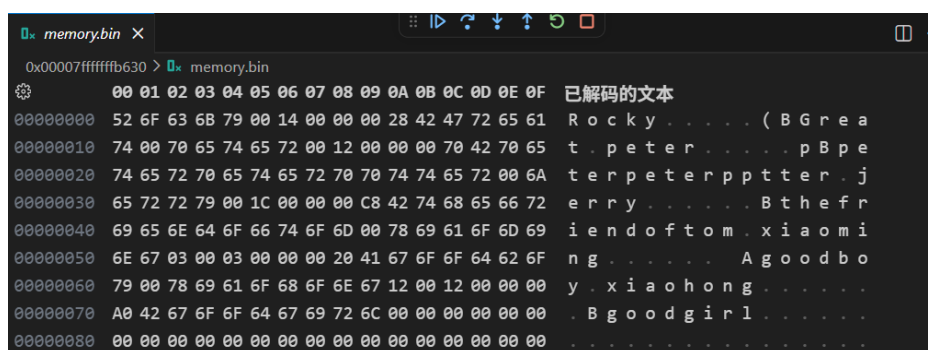


图 1-2 message 在调试器内存窗口中的十六进制表示

为验证压缩数据在内存中的真实排列,在调试器中观察 message 所在地址的内存窗口,如图1-2所示,可以看到前若干字节与控制台输出完全一致,且 ASCII 解码后能直接对应 Rocky、Great、peter 等字段内容。这说明字符串以字符流加结束符存放, short 与 float 按照机器字节序直接写入缓冲区。

```
1 1.abs 测试通过
2 2.negate 测试通过
3 3.bitAnd 测试通过
4 4.bitOr 测试通过
5 5.bitXor 测试通过
6 6.isTmax 测试通过
7 7.bitCount 测试通过
8 8.bitMask 测试通过
9 9.addOK 测试通过
10 10.byteSwap 测试通过
11 11.bang 测试通过
12 12.bitParity 测试通过
```

Listing 4 任务 2 自动测试结果摘录

```
1.abs 测试...
1.abs 测试通过

2.negate 测试...
2.negate 测试通过

3.bitAnd 测试...
3.bitAnd 测试通过

4.bitOr 测试...
4.bitOr 测试通过

5.bitXor 测试...
5.bitXor 测试通过

6.isTmax 测试...
6.sTmax 测试通过

7.bitCount 测试...
7.bitCount 测试通过

8.bitMask 测试...
8.bitMask 测试通过

9.addOK 测试...
9.addOK 测试通过

10.byteSwap 测试...
10.byteSwap 测试通过

11.bang 测试...
11.bang 测试通过

12.bitParity 测试...
12.bitParity 测试通过
```

图 1-3 任务 2 位运算函数自动测试结果截图

任务 2 的自动测试结果如图 1-3 所示。12 个函数均输出“测试通过”，说明自动对拍结果与标准实现一致。

1.5.2 结果分析

从任务 1 的结果可以看出，原始结构体数组共有 5 个元素，每个 `student` 结构体长度为 216 字节，因此压缩前总长度为 1080 字节。该长度并不只是字段有效数据之和，还包含了结构体内部由于对齐带来的填充空间。压缩后 `message` 的有效长度仅为 123 字节，相比原始存储减少了 957 字节，压缩后长度约为原来

的 11.39%，说明将结构体按字段顺序串行化能够显著消除对齐填充和未使用空间的影响。

从十六进制结果可以验证小端机器上的存储规律。前 6 个字节 52 6F 63 6B 79 00 对应字符串 "Rocky\0"；随后 14 00 对应年龄 20 的 `short` 编码；再后的 00 00 28 42 对应分数 42.0 的单精度浮点表示 0x42280000。其中符号位为 0，阶码为 10000100，尾数为 010100000000000000000000，与程序打印结果和内存窗口观察结果完全一致，说明编码分析正确。

解压后的 5 个学生信息与压缩前输出逐项一致，表明两种压缩函数虽然写入方式不同，但都遵循了统一的字段顺序，因此 `restore_student` 能够用同一种规则正确恢复数据。这也说明“字符串保留结束符、数值类型直接写入底层字节”的设计是可逆的。

任务 2 中 12 个函数全部通过自动对拍，说明补码运算、德摩根律、掩码分层统计和异或折叠等方法均实现了预期功能。尤其是 `bitCount`、`byteSwap` 和 `bitParity` 这类相对复杂的函数，能够在受限运算符条件下通过测试，表明位级化简思路是正确且可执行的。需要指出的是，本次随机测试每项仅执行 10 组样例，已经足以验证基本正确性，但若后续需要进一步增强可靠性，还可以扩大样本范围并增加极端边界值测试。

1.6 实验小结

1.6.1 理论与方法总结

本次实验将课堂中的“数据表示”与“位级运算”知识具体落实到了可执行程序中。任务 1 的关键理论基础是结构体布局、字节对齐、小端存储和 IEEE 754 浮点编码。通过把 `student` 结构体拆分为字符串字段、`short` 字段和 `float` 字段依次写入缓冲区，可以直观区分“逻辑上的数据内容”和“内存中的物理表示”。在此基础上，再按照完全相同的字段顺序进行反向读取，就能实现结构化数据的可逆压缩与恢复。这个过程说明，理解类型的底层字节表示，是掌握序列化、网络传输和二进制文件组织方式的基础。

任务 2 则主要体现了补码系统和布尔代数在机器级运算中的作用。很多看似依赖高级语义的操作，例如绝对值、逻辑非、溢出判断和奇偶校验，本质上都可以归结为符号位分析、按位组合和掩码筛选。实验中使用的德摩根律、异或消

去、分治统计和字节提取回填等方法，说明 CPU 级别的运算能力虽然基础，但通过合理组合仍然可以完成复杂功能。这一部分进一步加深了我对“高级语言表达最终会退化为位操作”的理解。

1.6.2 结果与收获总结

从实验结果看，任务 1 成功完成了 5 个学生信息的压缩、解压和内存数据对照，任务 2 成功完成了 12 个位运算函数的实现与自动测试，整体达到了实验要求。相比直接存储结构体数组，压缩后的数据长度显著下降，同时内存窗口与程序输出保持一致，这使得“字节序”“对齐”“浮点编码”等原本较抽象的概念变得可以直接观察和验证。位运算部分则让我更熟悉了如何在严格的运算符约束下构造等价表达式，并通过自动对拍提高调试效率。

在实现过程中，我也认识到程序健壮性与测试覆盖仍有进一步提升空间。例如，任务 1 当前默认输入格式合法，且备注字段不包含空格；任务 2 虽然已经加入随机测试和部分边界测试，但样本数量仍然有限。后续如果继续完善，可以为解压函数增加更严格的边界检查，为任务 1 补充更丰富的异常输入测试，为任务 2 增加更多极值用例和批量测试次数，从而使程序不仅“能运行”，而且在正确性和鲁棒性上更加充分。

实验 2：汇编和 C 的混合编程及优化

2.1 实验概述

1. 熟练掌握程序开发平台 (VS2019/Vs2022/Vs2023) 下 C 和汇编语言的混合编程方法；
2. 熟悉掌握常用机器指令的使用方法；
3. 掌握代码优化的基本方法，提高程序运行速度。

2.2 实验内容

任务 1 用 C 语言编写一个学生成绩管理程序，具有平均成绩计算、按平均成绩从高到低排序、显示学生成绩的功能

定义了结构 `student`，设有 `N` 个学生

```
1 struct student {  
2     char name[8];  
3     char sid[11]; // 如 U202315123  
4     short scores[8]; // 8 门课的分  
5     short average; // 平均分  
6 };
```

要求：

1. 第 0 个人姓名 (`name`) 为自己的名字，`sid` 为自己的学号；
2. 对于函数“计算平均成绩”、“按平均成绩从高到低排序”分别计时，显示两个函数的时间开销；
3. 显示排序前、排序后的学生信息（姓名、学号、8 门课的分、平均分），一个学生一行。
4. 对“计算平均成绩”、“按平均成绩排序”函数的 Debug 版、Release 版的运行效率进行对比，比较两个版本下的反汇编代码的差异，分析程序执行速度提高的原因。

任务 2 用汇编语言编写函数“计算平均成绩”，代替原来用 C 语言编写的函数

任务 3 对用汇编语言编写的“计算平均成绩”函数进行优化

任务 4 对排序函数进行算法优化

2.3 实验设计

2.3.1 解题思路分析

实验 2 围绕“用汇编语言替换并优化关键计算路径”展开，整体程序由 C 主控逻辑和 MASM 编写的平均分计算函数两部分组成。首先使用 C 语言完成学生信息初始化、成绩显示、排序与计时框架，使程序具备完整的数据流和可观测输出；随后将“计算平均成绩”这一热点函数从 C 迁移到汇编语言实现，并进一步针对固定 8 门课这一已知条件做展开优化，以比较不同实现方式对运行效率的影响。

从功能分解上看，任务 1 负责构造完整的学生成绩管理流程，任务 2 将平均分计算改写为汇编版本，任务 3 在汇编版本基础上继续减少循环控制和除法开销，任务 4 则将按平均分排序交给标准库 `qsort` 完成，避免手写低效排序。这样设计的原因在于：平均分计算属于规则固定、重复次数高、适合底层优化的数值处理任务，而排序则更适合复用库中成熟实现，两者分别对应“指令级优化”和“算法级优化”两条路线。

因此，本实验并不是单纯把 C 代码翻译成汇编代码，而是将程序拆分为“高层流程由 C 管理、性能关键段由汇编实现”的协同结构，再结合计时结果分析不同层次优化带来的收益。这种分层设计既保留了程序的可读性，又能直接观察机器级实现对执行效率的影响。

2.3.2 技术与方法

实验所采用的数据结构为 `student` 结构体，每个学生包含姓名、学号、8 门课程分数和平均分字段。程序以结构体数组作为核心存储载体，先批量初始化，再依次执行平均分计算、排序和显示。由于 `scores` 为定长数组，汇编代码可以直接依据结构体偏移访问每一门成绩，从而避免额外的间接寻址开销。

在计时方法上，程序同时给出了 `QueryPerformanceCounter` 和 `GetTickCount` 的示例。正式计时统一由宏 `Time_Anyfunc` 封装，并通过高精度性能计数器统计函数执行时间。相比 `GetTickCount`，这种方式更适合测量微小时间差，也更容易反映未优化版本与优化版本在平均分计算上的差异。

在汇编实现上，`computeAverageScore` 采用双层循环：外层遍历学生，内层遍历 8 门课程成绩并累加，最后通过 `idiv 8` 求出平均值。优化版

`computeAverageScore_improve` 利用课程数固定这一条件，直接展开 8 次取数与累加操作，并将除以 8 替换为右移 3 位，从而减少循环分支、索引维护和除法指令的开销。排序部分使用 `qsort` 配合比较函数按平均分降序排列，体现以现成高效算法替代基础实现的优化思路。

2.4 实验过程

2.4.1 环境准备

- 操作系统：Windows 11
- 开发环境：Visual Studio 2022
- 编译工具链：MSVC v145、MASM、Windows SDK 10.0
- 工程配置：同时提供 Debug/Release、Win32/x64 配置，其中 MASM 作为自定义生成步骤参与构建

2.4.2 具体实施步骤

1. 在 Visual Studio 中打开 汇编和 C 的混合编程及优化.slnx 工程，确认项目文件中已经启用 MASM 构建支持，并将 `main.c` 与汇编文件同时纳入构建系统。
2. 在 `main.c` 中定义 `student` 结构体和 10 个学生的结构体数组，编写初始化学生数据相关代码。其中第 0 个学生姓名设为 JunHao，学号设为 U202490042，其余学生按规则自动生成姓名、学号和 8 门课程随机分数。
3. 完成显示与排序逻辑。`displayStu` 负责逐行输出学生姓名、学号、8 门课程分数和平均分；`compareStudent` 与 `SortStudentScore` 负责调用 `qsort` 按平均分从高到低排序。这样可以在相同数据集上直接比较排序前后的结果是否满足实验要求。
4. 在汇编文件中实现第一个平均分函数 `computeAverageScore`。该版本使用外层学生循环和内层课程循环逐项累加成绩，再执行带符号除法求平均值，重点体现 C 与汇编混合编程的基本流程、结构体偏移访问方式以及寄存器保存规则。
5. 继续在同一汇编文件中实现优化版本 `computeAverageScore_improve`。由于每个学生固定只有 8 门课程，因此直接展开 8 次 `movzx + add` 操作，并用

`shr eax, 3` 替代 `idiv 8`, 从而减少循环判断和昂贵除法指令带来的时间消耗。

6. 使用 `QueryPerformanceCounter` 封装宏 `Time_Anyfunc` 对未优化平均分函数、优化后平均分函数以及排序函数分别计时, 并在控制台打印运行时间。运行程序后先输出计时结果, 再输出排序前与排序后的全部学生信息, 最后结合截图记录实验结果。
7. 对 `Debug` 与 `Release` 配置进行对比分析。虽然当前保留的输出截图来自一次具体运行结果, 但项目文件显示 `Release` 配置在分析部分重点讨论这些编译开关如何配合汇编优化进一步改善执行效率。

```
1 # 在 Visual Studio 中选择 Debug 或 Release 配置后直接生成并运行
2 # 工程同时编译 main.c 与 computeAverageScore.asm
3
4 # 典型执行流程
5 Build Solution
6 Start Without Debugging
```

Listing 5 实验 2 使用的构建与运行方式

```
1 computeAverageScore_improve proc uses edi ecx edx, sptr:dword, num:dword
2     mov edi, sptr
3     mov ecx, num
4 student_loop:
5     xor eax, eax
6     movzx edx, word ptr [edi].student.scores[0]
7     add eax, edx
8     movzx edx, word ptr [edi].student.scores[2]
9     add eax, edx
10    ; ... 中间 6 次同样展开 ...
11    movzx edx, word ptr [edi].student.scores[14]
12    add eax, edx
13    shr eax, 3
14    mov [edi].student.average, ax
15    add edi, type student
16    dec ecx
17    jnz student_loop
18    ret
19 computeAverageScore_improve endp
```

Listing 6 优化版平均分计算核心代码

2.5 实验结果

2.5.1 结果展示

表 2-1 实验 2 关键结果记录表

测试项目	测试场景	预期结果	实际结果
Test 1	输出结构体大小	结构体大小与对齐规则一致	38 字节
Test 2	未优化平均分计算时	能正确计算全部平均分	约 0.000400 ms
Test 3	优化版平均分计算时	时间不高于未优化版	约 0.000200 ms
Test 4	按平均分降序排序	排序后平均分单调不增	排序结果正确，最高分排在最前
Test 5	排序函数计时	能完成 10 个学生排序	约 0.017900 ms

实验 2 的控制台输出如图2-1所示。程序首先演示两种 Windows 计时接口的调用效果，然后输出结构体大小、未优化平均分函数计时、优化后平均分函数计时、排序前学生信息、排序耗时以及排序后的学生信息。

```

1 counter demo : 1.794700 毫秒
2 GetTickCount demo: 0 毫秒
3 38
4
5 计算平均成绩时间（未优化）：
6 Time_Anyfunc : 0.000400 毫秒
7 计算平均成绩时间（优化）：
8 Time_Anyfunc : 0.000200 毫秒
9
10 计算按平均成绩排序时间：
11 Time_Anyfunc : 0.017900 毫秒

```

Listing 7 实验 2 关键运行结果摘录

从排序前后的输出可以看到，学生 JunHao 的平均分为 85，始终位于排序结果首位；其后依次为平均分 63、54、51、50、45、45、43、43 和 32 的学生记录，

```

counter demo : 1.794700 毫秒
GetTickCount demo: 0 毫秒
38

计算平均成绩时间 (未优化) :
Time_Anyfunc : 0.000400 毫秒
计算平均成绩时间 (优化) :
Time_Anyfunc : 0.000200 毫秒
排序前的学生信息如下:
学生姓名: JunHao 学号:U202490042 八门课分数分别为:95 81 82 83 84 85 86 87 平均分为:85
学生姓名: Stu2 学号:U202400002 八门课分数分别为:83 19 84 1 67 74 77 6 平均分为:51
学生姓名: Stu3 学号:U202400003 八门课分数分别为:87 62 82 1 35 78 96 63 平均分为:63
学生姓名: Stu4 学号:U202400004 八门课分数分别为:79 9 19 63 45 21 69 40 平均分为:43
学生姓名: Stu5 学号:U202400005 八门课分数分别为:29 22 83 20 87 95 71 32 平均分为:54
学生姓名: Stu6 学号:U202400006 八门课分数分别为:76 24 28 47 63 52 33 25 平均分为:43
学生姓名: Stu7 学号:U202400007 八门课分数分别为:47 93 11 8 49 9 1 44 平均分为:32
学生姓名: Stu8 学号:U202400008 八门课分数分别为:87 42 68 38 71 1 78 22 平均分为:50
学生姓名: Stu9 学号:U202400009 八门课分数分别为:61 42 55 53 80 18 39 16 平均分为:45
学生姓名: Stu10 学号:U202400010 八门课分数分别为:68 45 11 32 51 17 89 49 平均分为:45
计算按平均成绩排序时间:
Time_Anyfunc : 0.017900 毫秒
排序后的学生信息如下:
学生姓名: JunHao 学号:U202490042 八门课分数分别为:95 81 82 83 84 85 86 87 平均分为:85
学生姓名: Stu3 学号:U202400003 八门课分数分别为:87 62 82 1 35 78 96 63 平均分为:63
学生姓名: Stu5 学号:U202400005 八门课分数分别为:29 22 83 20 87 95 71 32 平均分为:54
学生姓名: Stu2 学号:U202400002 八门课分数分别为:83 19 84 1 67 74 77 6 平均分为:51
学生姓名: Stu8 学号:U202400008 八门课分数分别为:87 42 68 38 71 1 78 22 平均分为:50
学生姓名: Stu10 学号:U202400010 八门课分数分别为:68 45 11 32 51 17 89 49 平均分为:45
学生姓名: Stu9 学号:U202400009 八门课分数分别为:61 42 55 53 80 18 39 16 平均分为:45
学生姓名: Stu6 学号:U202400006 八门课分数分别为:76 24 28 47 63 52 33 25 平均分为:43
学生姓名: Stu4 学号:U202400004 八门课分数分别为:79 9 19 63 45 21 69 40 平均分为:43
学生姓名: Stu7 学号:U202400007 八门课分数分别为:47 93 11 8 49 9 1 44 平均分为:32
    
```

图 2-1 实验 2 程序输出截图

说明排序结果满足“按平均分从高到低排列”的实验要求。截图中还展示了每个学生 8 门课的具体分数，说明程序不仅完成了计时，还完成了完整的数据展示功能。

2.5.2 结果分析

从结果上看，实验 2 已经完成了任务书要求的主要功能。首先，程序成功输出了 `sizeof(student)=38`，这说明当前结构体由 `char[8]`、`char[11]`、对齐填充、`short[8]` 和 `short average` 共同组成，验证了汇编文件中通过 `align 2` 控制字段对齐的设计是有效的。结构体大小与字段布局一致，也为后续汇编代码使用固定偏移访问 `scores` 和 `average` 提供了前提。

在平均分计算部分，未优化版本耗时约为 0.000400 ms，优化版本耗时约为 0.000200 ms，优化后时间约下降 50%。这一结果与代码结构高度一致：未优化版本使用了课程循环和 `idiv` 指令，存在更多分支控制与整数除法开销；优化版本将 8 次取数完全展开，并把除以 8 转化为右移 3 位，因此指令路径更短、流水线更稳定。虽然绝对时间很短，但相对差异已经能够体现汇编级展开和位移优化的效果。

排序部分耗时约为 0.017900 ms，明显高于单次平均分计算，这是因为 `qsort`

需要多次比较与交换整个结构体记录。尽管如此，排序结果从 85 分一直降到 32 分，说明比较函数 `return s2->average - s1->average;` 能够正确实现降序逻辑。结合输出截图可以看到，排序前后学生的课程分数没有发生改变，仅记录顺序发生调整，说明排序操作保持了数据项的完整性。

对于 Debug 与 Release 的差异，当前工程文件表明 Release 的配置会减少额外调试检查、改善函数内联和链接期优化效果。因此在相同代码下，Release 版本理论上会比 Debug 版本具有更低的调用开销和更紧凑的目标代码。再叠加实验中手工编写的汇编优化，最终性能提升并不只来自汇编本身，也来自编译配置对整个程序执行路径的优化。

2.6 实验小结

2.6.1 理论与方法总结

本次实验系统地展示了 C 语言与汇编语言混合编程的基本方法。C 语言更适合组织程序框架、管理数据结构和控制整体流程，而汇编语言更适合针对热点计算做细粒度优化。通过把学生信息初始化、显示和排序保留在 C 层，把平均分计算迁移到 MASM 层，我进一步理解了高级语言与机器级代码之间的职责分工，也熟悉了在工程中同时编译 .c 和 .asm 文件的实际方法。

从优化方法来看，实验同时体现了两类典型思想。一类是指令级优化，即在 `computeAverageScore_improve` 中展开固定次数循环、消除除法、减少索引维护；另一类是算法级优化，即在排序任务中直接采用 `qsort` 这样的成熟库实现，而不是停留在低效的手写简单排序。前者强调“单次计算路径更短”，后者强调“总体操作次数更少”，两者共同构成了程序性能优化的基础思路。

2.6.2 结果与收获总结

从实验结果看，实验 2 已经完成了学生成绩管理、平均分计算、排序显示、函数计时和汇编优化等要求。尤其是优化版平均分函数相较未优化版进一步缩短了执行时间，说明结合问题特征进行手工汇编优化是有效的。与此同时，排序前后的输出也验证了数据处理流程的正确性，使实验不仅停留在“能运行”，而且能够通过输出结果和计时结果双重证明实现的有效性。

本次实验带来的最大收获有两点。第一，我对结构体在内存中的布局、字段

偏移和寄存器约定有了更具体的理解，能够把这些底层概念直接应用到汇编代码编写中。第二，我更清楚地认识到性能优化不是单一手段，而是代码结构、指令选择、算法选型和编译配置共同作用的结果。后续如果继续完善该实验，可以补充 Debug/Release 两种配置下的实际测量数据，并在排序部分进一步比较不同排序算法的时间差异，从而使实验分析更加完整和量化。

实验 3：机器级语言理解 (二进制炸弹拆除)

3.1 实验概述

1. 熟悉可执行程序在机器级上的输入校验流程，理解字符串比较、整数递推、跳转表、递归编码和链表重排等典型实现方式；
2. 掌握 gdb、objdump、nm 等工具对二进制程序进行反汇编、单步调试和数据段分析的方法；
3. 能够结合静态分析与动态调试，从汇编和内存信息中恢复程序逻辑并构造满足约束的输入。

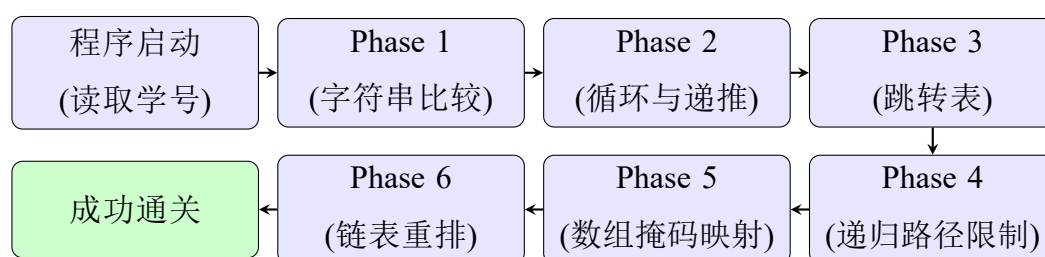


图 3-1 二进制炸弹依次经过的六个处理阶段控制流示意图

3.2 实验内容

操作说明

整个程序包含有：bomb.c support.c phases.c support.h。在老师手上有 phases.c，但同学们没有该源程序，而只有 phases.o。

提供给学生的包：bomb.c support.c phases.o support.h

由学生生成执行程序：

```
1 gcc -g -o bomb -D U* bomb.c support.c phases.o
```

其中 * 为学号的最后一位。例如：学号为 U202415001，则使用命令

```
1 gcc -g -o bomb -D U1 bomb.c support.c phases.o
```

运行程序：

```
1 ./bomb
```

下面给出了三关通过的效果。

```
1 root@LAPTOP-CJLSTBTI:/home/binary_bomb#
2 root@LAPTOP-CJLSTBTI:/home/binary_bomb# gcc -c -D PROBLEM phases.c -o phases
   .o
3 root@LAPTOP-CJLSTBTI:/home/binary_bomb# gcc -g -o bomb -D U3 bomb.c support.
   c phases.o
4 root@LAPTOP-CJLSTBTI:/home/binary_bomb# ./bomb
5 Input your Student ID :
6 U202215123
7 welcome U202215123
8 You have 6 bombs to defuse!
9 Gate 1 : input a string that meets the requirements.
10 Computer System Foundation.
11 Phase 1 passed!
12 Gate 2 : input six intergers that meets the requirements.
13 3 2 5 7 12 19
14 Phase 2 passed!
15 Gate 3 :
16 input 2 intergers.
17 1 304
```

提醒:

1. 一定要输入自己的学号，运行结果中会显示学号的。
2. 在生成程序时，一定要带编译开关 `-D U*`；* 为学号的最后一位数字。

3.3 实验设计

3.3.1 解题思路分析

实验3的核心任务不是修改源程序，而是在缺失 `phases.c` 源代码的情况下，仅依据 `phases.o`、可执行程序行为以及反汇编结果恢复6个关卡的输入约束。为此，本实验采用“先建立可执行程序，再结合静态分析和动态调试逐关提取约束”的方案。整体上，先使用 `bomb.c`、`support.c` 与 `phases.o` 重新链接出可执行程序 `bomb`，再借助 `gdb` 观察关键比较点处的寄存器、栈变量和跳转目标，同时配合 `objdump` 和数据段内容恢复隐藏常量表与链表结构，最终得到全部通关答案。

之所以采用这种方案，是因为不同关卡对应的控制逻辑并不相同。第一关本质上是固定字符串比较，可以直接通过反汇编和只读数据段定位目标串。第二关

和第三关与学号有关，适合在 `cmp` 指令前读取程序期望值，逐步修正输入。第四关需要理解递归函数 `func4` 的返回值编码规则，属于“先读取目标值，再反推出输入”的类型。第五关依赖字符低 4 位映射表，静态查看 `array.0` 与目标串即可构造答案。第六关则要从数据段恢复链表节点值，再按降序重新排列节点编号。换言之，本实验不是简单“猜答案”，而是针对每一关的机器级结构选择最合适的逆向路径。

因此，实验 3 的总体设计强调两点：一是把程序执行过程拆解为“输入读取、约束计算、最终比较”三个阶段，逐步定位答案来源；二是根据关卡特征在动态调试和静态推导之间切换，避免陷入纯暴力试错。这样既能提高拆弹效率，也能更直接地对应课程中关于控制流、数据流和目标代码结构的知识点。

3.3.2 技术与方法

实验中主要使用的技术手段包括 `gdb` 单步调试、`objdump` 反汇编、`nm` 符号查看和数据段静态分析。`gdb` 主要用于在关键比较指令或函数入口处设置断点，观察寄存器和栈上输入变量的内容，从而直接读出程序内部计算得到的期望值。`objdump -drwC` 用于恢复各关卡的完整控制流。`objdump -s -j .rodata` 和 `objdump -s -j .data` 用于提取固定字符串、字符映射表与链表节点值。`nm -n` 则辅助定位函数和数据符号。

从关卡结构上看，六关分别对应不同类型的机器级约束。第一关使用 `strings_not_equal` 比较输入串与固定常量。第二关读取 6 个整数，并检查前两项是否等于学号中指定字符的数字值、后四项是否满足递推关系。第三关先比较第 1 个整数是否等于学号某一位数字，再经由跳转表为第 2 个整数生成目标常量。第四关调用递归函数 `func4` 对搜索路径进行编码，并要求两个输入同时匹配目标值 7。第五关使用字符低 4 位作为下标访问数组 `array.0`，再将映射结果与字符串 `bruins` 比较。第六关则把 6 个整数解释为链表节点位置，按输入顺序重连链表，并要求新链表中节点值单调不增。

在数据结构方面，实验 3 涉及两个最关键的静态对象。其一是第五关中的字符映射表 `"maduiersnfotvbyl"`，它决定了输入字符经低 4 位映射后得到的输出串；其二是第六关中的 `listNode` 链表结构，节点中包含数值、编号和下一节点指针，只有将节点值按降序排列后，重新连接得到的节点序号序列才能通过最终比较。通过这些对象可以看出，二进制程序的输入校验本质上仍然依赖常量表、

控制流分支和内存中的结构化数据，只是这些信息需要从机器级表示中恢复出来。

3.4 实验过程

3.4.1 环境准备

- 操作系统：Ubuntu on WSL
- 编译工具：GCC
- 调试与分析工具：GDB、objdump、nm
- 实验文件：bomb.c、support.c、phases.o、support.h 以及辅助反汇编文件

3.4.2 具体实施步骤

1. 根据任务书要求，使用 `gcc -g -o bomb -D U2 bomb.c support.c phases.o` 重新生成可执行程序。由于本人学号 U202490042 的最后一位为 2，因此编译时选择 `-D U2`，保证程序中与学号相关的逻辑分支和本人的输入一致。
2. 首先对 bomb.c 做整体流程阅读，确认程序运行时依次要求输入学号、第一关字符串、第二关 6 个整数、第三关 2 个整数、第四关“2 个整数和一个字符串”、第五关字符串以及第六关 6 个整数。这样可以先建立完整的输入顺序，再进入逐关调试。
3. 第一关通过静态分析获得答案。查看 `phase_1` 的汇编可以发现该关最终调用 `strings_not_equal` 比较输入与一个固定字符串，因此结合反汇编和只读数据内容得到答案为 ABI Application Binary Interface.。
4. 第二关采用“占位输入 + 比较点读取期望值”的方式处理。先输入 0 0 0 0 0 0 进入逻辑，再在 `phase_2` 的关键 `cmp` 指令前观察寄存器。由汇编可知前两个数分别取自 `studentid[9] - '0'` 和 `studentid[8] - '0'`，结合本人学号可得前两项为 2 和 4；后续 4 项满足递推式 `nums[i] = nums[i-1] + nums[i-2] + 1`，因此推得第二关答案为 2 4 7 12 20 33。
5. 第三关同样使用断点观察和静态跳转表分析。第一层比较要求第一个整数等于 `studentid[8] - '0'`，因此第一个数为 4；随后跳转表分支把目标常量

写入局部变量，其中对应 $x=4$ 的分支写入 `0x11c`，即十进制 284，故第三关答案为 4 284。

6. 第四关通过分析递归函数 `func4` 完成。该函数本质上是对区间 $[0,14]$ 上的二分查找路径进行编码，而外层 `phase_4` 把目标常量固定为 7。由 `func4(x,0,14)=7` 可反推出 $x=14$ ，同时第二个输入也必须等于 7，因此第四关答案为 14 7。
7. 第五关主要依赖静态数据段。汇编显示程序先检查输入长度是否为 6，然后对每个字符执行 `input[i] & 0xf`，以结果为下标访问字符表 `"maduiersnfotvbyl"`，最后与目标字符串 `"bruins"` 比较。只要输入字符低 4 位依次对应 13,6,3,4,8,7 即可，通过构造可打印字符得到答案 `m63487`。
8. 第六关通过静态恢复链表节点值求解。查看数据段后可知 6 个节点的值依次为 281, 907, 322, 121, 824, 528。程序把输入的 6 个数字解释为“取第几个节点”，并按输入顺序重新链接；最后要求新链表中节点值非增。因此将节点按值从大到小排序后，对应的位置序号为 2 5 6 3 1 4。
9. 在得到 6 关答案后，重新运行程序并按顺序输入：学号 U202490042、第一关字符串、第二关六元组、第三关二元组、第四关二元组、第五关字符串和第六关六元组，最终所有关卡全部通过，并保存控制台截图用于结果展示。

```
1 # 生成可执行程序
2 gcc -g -o bomb -D U2 bomb.c support.c phases.o
3
4 # 运行程序
5 ./bomb
6
7 # 反汇编与静态查看
8 objdump -drwC bomb > bomb.asm
9 objdump -s -j .rodata bomb
10 objdump -s -j .data bomb
11 nm -n bomb
12
13 # gdb 动态调试
14 gdb -q ./bomb
15 break phase_2
16 break phase_3
17 break phase_4
18 run
```

Listing 8 实验 3 中使用的典型命令

```
1 int func4(int x, int low, int high) {
2     int mid = low + (high - low) / 2;
3     if (x < mid) {
4         return 2 * func4(x, low, mid - 1);
5     }
6     if (x > mid) {
7         return 2 * func4(x, mid + 1, high) + 1;
8     }
9     return 0;
10 }
```

Listing 9 第四关递归函数逻辑抽象

3.5 实验结果

3.5.1 结果展示

表 3-1 实验 3 各关卡答案与依据

关卡	正确输入	关键依据
Phase 1	ABI Application Binary Interface.	固定字符串比较
Phase 2	2 4 7 12 20 33	学号取位加递推关系
Phase 3	4 284	学号取位加跳转表常量
Phase 4	14 7	func4 路径编码和目标值 7
Phase 5	m63487	低 4 位映射到 bruins
Phase 6	2 5 6 3 1 4	链表节点值按降序重排

实验 3 最终输入序列如下。该序列依次包含学号和六个关卡的答案，按顺序输入后可一次性拆除全部炸弹。

```
1 U202490042
2 ABI Application Binary Interface.
3 2 4 7 12 20 33
4 4 284
5 14 7
```

6	m63487
7	2 5 6 3 1 4

Listing 10 实验 3 最终通关输入

实验 3 程序运行结果如图3-2所示。截图显示程序成功通过 6 个关卡，并最终输出 Congratulations! You have passed all the tests!, 说明所有输入均满足程序内部的机器级约束。

```
(sage) kms12425@LAPTOP-3BF1R810:~/my-wsl-project/Foundations of Computer Systems/LAB_3$ gcc -g -o bomb -D U2 bomb.c support.c phases.o
(sage) kms12425@LAPTOP-3BF1R810:~/my-wsl-project/Foundations of Computer Systems/LAB_3$ ./bomb
Input your Student ID :
U202490042
welcome U202490042
You have 6 bombs to defuse!
Gate 1 : input a string that meets the requirements.
ABI Application Binary Interface.
Phase 1 passed!
Gate 2 : input six integers that meets the requirements.
2 4 7 12 20 33
Phase 2 passed!
Gate 3 : input 2 integers.
4 284
Phase 3 passed!
Gate 4 : input 2 integers and a string.
14 7
Phase 4 passed!
Gate 5 : input a string.
m63487
Phase 5 passed!
Gate 6 : input 6 integers.
2 5 6 3 1 4
Phase 6 passed!
Congratulations! You have passed all the tests!
```

图 3-2 实验 3 全部关卡通过后的程序输出截图

从最终运行结果可以看到，第一关到第六关都依次打印了 Phase x passed!。其中第二关显示的六元组为 2 4 7 12 20 33，第三关为 4 284，第四关为 14 7，第五关字符串为 m63487，第六关为 2 5 6 3 1 4，与静态分析和动态调试得到的结果完全一致。

3.5.2 结果分析

实验 3 的结果表明，六个关卡的输入并不是彼此独立的“谜语”，而是分别对应六类常见的机器级程序结构。第一关是最直接的固定字符串比较；第二关引入了基于学号字符的参数化递推关系；第三关体现了 switch/jump-table 结构在汇编层的实现形式；第四关展示了递归函数如何把搜索路径编码到返回值中；第五关说明字符映射问题常常隐藏在数组索引和掩码运算里；第六关则把链表重连与顺序检查结合起来，要求同时理解数据结构和控制流。这些关卡覆盖了字符串、整数、分支、递归、表驱动和链表等多个层面的程序行为，因此非常适合作为机器级语言理解训练。

从求解过程看，动态调试和静态分析各有优势。第二关和第三关中，利用 gdb 在 cmp 指令前直接读取期望值，可以迅速确认学号相关的目标数字；而第五

关和第六关若仅靠单步调试会比较繁琐，直接查看 `.rodata` 和 `.data` 中的映射表、目标串和节点值反而更高效。这说明在逆向分析中，正确选择分析入口比盲目增加断点更重要。只有把“控制流恢复”和“数据对象恢复”结合起来，才能真正高效地还原程序逻辑。

此外，实验结果也说明二进制程序虽然缺少高级语言源代码，但其语义并没有消失，而是被编码进了指令序列、常量表和内存布局中。例如第二关的递推式、第四关的 `func4`、第五关的映射数组以及第六关的链表节点值，都可以在汇编和数据段中被明确定位。也正因为如此，最终能够在不拥有 `phases.c` 的前提下完整推导出所有关卡答案，并通过截图验证全部通关。

3.6 实验小结

3.6.1 理论与方法总结

本次实验把课堂中的机器级程序表示知识具体落实到了实际的逆向分析过程中。通过拆弹任务，我进一步理解了函数调用约定、寄存器与栈帧布局、条件跳转、跳转表、递归返回值编码、数组索引和链表结构在汇编层的表现形式。与直接阅读 C 源代码相比，二进制分析要求先从指令和内存中恢复程序语义，再把这些零散信息组织成完整的逻辑约束，这一过程本身就是对机器级语言理解能力的系统训练。

从方法上看，本实验最重要的经验是“按关卡类型选择工具”。面对比较指令明确、期望值就存在寄存器或栈变量中的情况，动态调试效率最高；面对字符映射表、目标字符串或链表节点值这类稳定数据对象时，静态查看 `.rodata`、`.data` 和符号表更直接。两类方法并不是互相替代，而是互相补充：动态调试负责抓住关键时刻的运行状态，静态分析负责恢复长期存在的数据和结构，两者结合后才能快速而可靠地完成二进制推理。

3.6.2 结果与收获总结

从实验结果看，本次实验已成功拆除 6 个关卡，并通过最终截图验证程序完整输出 `Congratulations! You have passed all the tests!`。这说明基于本人学号 U202490042 推导出的整套答案是正确的，也说明所采用的静态分析与动态调试流程能够稳定恢复程序内部约束，而不是依赖偶然猜测。

本次实验最大的收获在于，我对“源代码缺失时如何理解程序”有了更具体的认识。过去更熟悉的是从 C 代码到汇编代码的正向阅读，而这次必须从汇编、符号和数据段反向推回程序语义，因此对 `cmp`、`lea`、条件跳转、栈变量偏移以及内存对象布局的理解都更扎实了。同时，我也意识到不同类型关卡适合不同解法，不能把所有问题都用同一种调试套路解决。后续若继续完善该实验，可以把每一关的关键断点、寄存器值和数据段截图进一步整理成更细致的证据链，使实验报告在“答案正确”之外，也能更完整地展示推导过程。

实验 4：缓冲区溢出攻击

4.1 实验概述

本实验围绕课程提供的缓冲区炸弹程序 `bufbomb` 展开，目标是在不修改源程序逻辑的前提下，仅通过构造输入字节串改变 `getbuf` 的返回行为。实验重点不在于“猜测字符串”，而在于结合函数调用约定、栈帧布局、小端序存储和控制流转移过程，分析缓冲区溢出为什么能够发生、攻击载荷应当如何组织，以及不同攻击目标对栈数据伪造程度的要求有何区别。

实验环境采用 Ubuntu、GCC 和 GDB。编译时关闭栈保护与位置无关执行，并根据本人学号 U202490042 的最后一位选择宏 `-DU2`，以保证 `getbuf` 的局部变量布局与调试分析一致。实验中主要使用 `gdb` 观察 `buf`、`rbp`、返回地址和目标函数入口地址，使用十六进制输入文件组织攻击载荷，再通过程序输出验证攻击是否成功。

4.2 实验内容

本次实验的总体内容是利用 `getbuf` 中对固定长度缓冲区 `buf` 的无边界复制漏洞，构造十六进制攻击串覆盖栈上的关键控制数据，从而改变函数返回后的执行路径。根据任务难度，实验分为两个阶段：第一阶段 `smoke` 仅要求覆盖返回地址并跳转到目标函数；第二阶段 `fizz` 不仅要劫持控制流，还要让目标函数读到正确参数，因此需要进一步伪造栈帧中的局部数据布局。

```
1 int getbuf(char *src, int len)
2 {
3     char buf[NORMAL_BUFFER_SIZE];
4     Gets(buf, src, len);
5     return 1;
6 }
```

Listing 11 缓冲区溢出漏洞对应的关键代码

4.2.1 阶段 1: `smoke` 返回地址覆盖攻击

1. 任务描述：本阶段要求在 `getbuf` 执行 `ret` 时，不返回 `test` 中原调用点，而是直接跳转到 `smoke` 函数入口，从而输出 `Smoke!: You called smoke()`。其本质

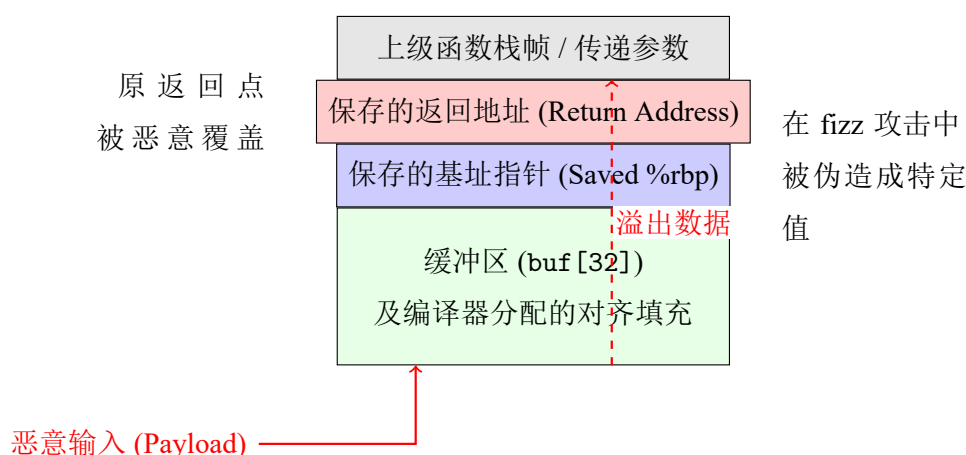


图 4-1 缓冲区溢出攻击导致栈帧内控制信息被覆盖的原理示意图

是覆盖保存的返回地址，使其变为 smoke 的入口地址。

2. 实验设计：该阶段只涉及最基本的控制流劫持，因此总体思路是先在 getbuf 建立栈帧后测量 buf 起始地址到保存返回地址的真实偏移，再查询 smoke 的入口地址，并将目标地址按 x86_64 小端序写入攻击串尾部。由于 buf 的名义长度虽然为 32 字节，但在 -DU2 条件下，函数内还存在额外局部数组和编译器对齐空间，所以必须通过调试而不是经验估计来确认偏移量。

3. 实验过程：首先使用以下命令重新编译程序，关闭栈保护并固定代码地址：

```

1 gcc -g -fno-stack-protector -no-pie -DU2 -fcf-protection=none -z execstack \
2   verinfo.c bufbomb.c buf.c -o bufbomb
3
4 gdb ./bufbomb
5 (gdb) b getbuf
6 (gdb) run U202490042 smoke_hex.txt 0
    
```

Listing 12 阶段 1 使用的编译与调试命令

在 getbuf 入口停下后，先单步越过函数序言，再查看 buf 与返回地址位置。本次调试得到返回地址距 buf 起始地址的偏移为 88 字节，smoke 的入口地址为 0x401445。因此攻击串的前 88 个字节全部用 0x41 填充，最后 8 个字节写入 smoke 地址的小端序 45 14 40 00 00 00 00 00。最终攻击文件内容如下：

```

1 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
2 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
3 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
4 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
5 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
6 41 41 41 41 41 41 41 41 45 14 40 00 00 00 00
    
```

Listing 13 阶段 1 最终攻击串 smoke_hex.txt

最后使用 `./bufbomb U202490042 smoke_hex.txt 0` 运行程序，观察是否转入 `smoke`。

4. 实验结果：程序成功输出 `Smoke!: You called smoke()`，说明返回地址已经被精确改写，`ret` 指令取出的不再是原返回点，而是 `smoke` 的入口地址。该结果表明：只要攻击者能够覆盖保存的返回地址，即使不修改任何源代码，也能直接改变程序控制流。这一阶段验证了缓冲区溢出的最基本破坏方式，即“覆盖返回地址并劫持执行路径”。

4.2.2 阶段 2: fizz 伪造栈帧攻击

1. 任务描述：本阶段要求 `getbuf` 返回后进入 `fizz` 的成功路径，并正确打印 `cookie` 值。与 `smoke` 不同，`fizz` 会比较参数 `val` 与全局变量 `cookie` 是否相等，因此仅把返回地址改成 `fizz` 入口并不能通过检查，必须让 `fizz` 在执行比较时读到正确的值。

2. 实验设计：在 `x86_64` 调用约定下，`fizz` 的第一个参数通过 `edi` 传递，函数入口还会执行参数落栈操作。如果直接跳到函数开头，寄存器中的参数无法由当前攻击串可靠控制。因此本阶段采用“跳过参数落栈指令 + 伪造 `rbp` + 在栈上预埋 `cookie`”的方案：让 `getbuf` 返回到 `fizz` 内部地址 `0x40146d`，避开用 `edi` 覆盖局部变量的语句；同时把保存的 `rbp` 改写成伪造值，使 `fizz` 随后访问 `[rbp-0x4]` 时，恰好读到攻击串中预先写入的 `cookie`。

3. 实验过程：先在 `gdb` 中反汇编 `fizz`，确认入口地址为 `0x401462`，内部合适的落点地址为 `0x40146d`。然后继续在 `getbuf` 中测量栈布局。实验中测得 `buf` 起始地址为 `0x7fffffffce10`，保存的 `rbp` 距 `buf` 偏移 80 字节，返回地址距 `buf` 偏移 88 字节。为了让 `fizz` 访问 `[rbp-0x4]` 时读到正确值，将伪造的 `rbp` 设为 `0x7fffffffce60`，于是 `cookie` 应写在偏移 76 至 79 字节处。程序运行时打印的 `cookie` 为 `0x0c11c0ba`，其小端序字节为 `ba c0 11 0c`。最终攻击串由 76 个填充字节、4 字节 `cookie`、8 字节伪造 `rbp` 和 8 字节返回地址组成：

```
1 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
2 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
3 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
```

```
4 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
5 41 41 41 41 41 41 41 41 41 41 41 41 ba c0 11 0c
6 60 ce ff ff ff 7f 00 00 6d 14 40 00 00 00 00 00
```

Listing 14 阶段 2 最终攻击串 fizz_probe.txt

由于该解法依赖稳定的栈地址，验证时需要关闭地址随机化，例如使用 `setarch $(uname -m) -R ./bufbomb U202490042 fizz_probe.txt 1`，或在 `gdb` 中开启 `set disable-randomization on` 后运行。

4. 实验结果：程序成功输出 `Fizz!: You called fizz(0xc11c0ba)`，说明控制流已从 `getbuf` 正确转入 `fizz` 内部比较逻辑，且 `fizz` 读取到的 `val` 与 `cookie` 完全一致。与阶段 1 相比，本阶段不再只是单纯改写返回地址，而是进一步利用溢出伪造了新的栈帧语义，使目标函数把攻击者布置的数据当作合法局部变量读取。这说明缓冲区溢出不仅能改变跳转目标，还能联动破坏函数上下文和参数语义，其危害明显高于最简单的返回地址覆盖。

4.3 实验小结

本次实验以 `bufbomb` 为对象，完成了 `smoke` 与 `fizz` 两个阶段的缓冲区溢出攻击练习。在第一阶段中，我亲眼看到，只要将保存的返回地址悄悄替换掉，程序就会乖乖跳到我们指定的地方去执行；第二阶段则更进一步，不仅要骗过程序的执行路径，还要让目标函数误以为自己收到了“合法”的参数，这让我深刻感受到缓冲区溢出的威力远不止于让程序崩溃那么简单。

回顾整个实验过程，我发现自己用到了许多课上学过的知识：函数调用时的栈帧结构、小端序的字节存储方式、`gdb` 的断点调试，还有反汇编代码的阅读方法。以前这些知识在我眼里不过是一些孤立的概念，而这次实验把它们串联在了一起。测量 `buf` 与返回地址之间的偏移、找到 `smoke` 和 `fizz` 的入口地址、把 `cookie` 藏进精心构造的输入文件……每一步都让我更真切地体会到：数据和代码在内存里本质上没有区别，程序信任什么，攻击者就能伪造什么。

通过这次实验，我有三点收获让我印象深刻。第一，栈帧不再是课本上的示意图，而是真实存在于内存某个地址处的字节序列，我可以用调试器直接看到它、修改它。第二，攻击并不神秘，它不过是把程序自己的运行规则用在了出乎意料的地方。第三，也是最重要的一点，安全问题的根源往往不是技术有多复杂，而是程序员在写代码时忽略了“用户输入不可信”这一基本原则。如果后续还有

机会，我很想继续完成 bang 和 boom 阶段，亲自验证一下开启栈保护和地址随机化之后攻击为什么会失效，把”怎么攻”和”怎么防”这两件事放在一起对比着理解。

实验 5：ELF 文件与程序链接

5.1 实验概述

本实验围绕 ELF 目标文件和程序链接过程展开，目标不是单纯完成若干命令操作，而是把“源程序如何逐步变成可执行文件”这一过程拆解为多个可观测阶段：先从 C 程序到汇编程序，再到可重定位目标文件，再到可执行文件，最后进一步比较静态链接与动态链接在文件结构、地址解析和函数定位方式上的差异。通过对同一份源程序在不同编译阶段产生的文件进行对照，可以把课堂中关于节、符号表、重定位、PLT/GOT 和共享库装载的抽象概念对应到具体的字节、地址与指令上。

本次实验采用 Ubuntu on WSL 环境，主要工具包括 GCC、readelf、objdump、nm、hexdump 和 ldd。根据本人学号 U202490042 的最后一位数字 2，实验中统一使用 `-D U2` 作为编译开关，并将 `lianjie experiment/main.c` 中的学号字符串同步修改为本人的学号。整体任务分为五个阶段：阶段 1 分析汇编层表示方式，阶段 2 研究可重定位目标文件结构，阶段 3 研究可执行文件中的最终地址与重定位结果，阶段 4 比较静态链接与动态链接生成程序的差异，阶段 5 分析动态链接库函数如 `printf` 的定位方法。

5.2 实验内容

本次实验的总体内容是对同一组源程序在“汇编程序、可重定位目标文件、可执行文件、静态链接文件、动态链接文件”这五种阶段性产物中分别观察其代码、数据、符号和地址组织方式。为了便于分析，实验先将 `lianjie experiment/main.c` 中的学号字符串修改为 U202490042，并将内联汇编中的重复次数改为学号末位对应的 2，之后统一在各任务中使用 `-D U2` 编译。实验的五个阶段与核心关注点如表 5-1 所示。

表 5-1 实验 5 五个阶段的目标与分析重点

阶段	目标	核心关注点
阶段 1	从 C 程序到汇编程序	全局变量、局部变量和函数调用在汇编层的表达形式
阶段 2	可重定位目标文件解析	节头表、符号表、重定位节以及数据节之间的对应关系
阶段 3	可执行目标文件解析	最终地址、节合并顺序、符号解析与重定位完成结果
阶段 4	静态链接与动态链接比较	文件大小、动态段、共享库依赖和函数调用路径差异
阶段 5	动态库函数定位方法	printf 从未定义符号到 PLT/GOT 定位的全过程

```

1 # 任务1: 从 C 到汇编
2 gcc -S -fverbose-asm -D U2 main.c -o main.s
3
4 # 任务2: 生成可重定位目标文件
5 gcc -c -g -D U2 main.c -o main.o
6
7 # 任务3: 生成两个链接顺序不同的可执行文件
8 gcc -g -D U2 main.c sub.c -o main_sub
9 gcc -g -D U2 sub.c main.c -o sub_main
10
11 # 任务4: 生成动态链接版与静态链接版
12 gcc -g -D U2 main.c sub.c -o main_sub_dyn
13 gcc -g -static -D U2 main.c sub.c -o main_sub_static

```

Listing 15 实验 5 中统一使用的典型编译命令

5.2.1 阶段 1: 从 C 程序到汇编语言程序

1. 任务描述: 本阶段要求使用 `gcc -S -fverbose-asm -D U2 main.c -o main.s` 生成汇编文件, 并结合具体变量与具体语句, 分析全局变量、局部变量和函数调用在汇编程序中的表示形式, 说明为什么有些对象进入 `.data` 段, 有些对象进入 `.bss` 或 `.rodata` 段。

2. 实验设计：这一阶段的重点是建立“高级语言语义”和“汇编层组织形式”之间的一一对应关系。实验选择三个最有代表性的切入点：一是比较 `init_gstr1` 与 `uninit_gstr2`，观察已初始化和未初始化全局变量分别如何进入 `.data` 与 `.bss`；二是用 `fadd(init_gv1, lv1)`、`fsub(50,30)` 等调用语句说明参数寄存器、返回值寄存器和 `call` 指令的配合；三是比较全局变量通过 `symbol(%rip)` 访问、局部变量通过 `offset(%rbp)` 访问的差异，从而解释不同存储期对象在汇编层的定位方式。

3. 实验过程：实验首先在 `lianjie experiment/main.c` 中将学号字符串改为 `U202490042`，并把内联汇编的重复次数调整为 2。随后执行 `gcc -S -fverbose-asm -D U2 main.c -o main.s` 生成 `lianjie experiment/main.s`。在分析过程中重点观察以下几类典型片段：一是 `init_gstr1`、`init_gv1`、`init_garr1` 在 `.data` 中使用 `.string`、`.long` 等伪指令初始化；二是 `uninit_gstr2`、`uninit_gv2` 等变量在 `.bss` 中通过 `.zero` 预留空间；三是字符串常量以 `.LCx` 标签形式进入 `.rodata`；四是函数调用通过将参数依次装入 `%edi`、`%esi`、`%rdi` 等寄存器，再执行 `call` 指令完成控制转移。

4. 实验结果：分析结果表明，已初始化全局变量通常进入 `.data` 段，未初始化全局变量通常进入 `.bss` 段，字符串常量通常进入 `.rodata` 段，而带地址初值的全局指针变量则进入带重定位属性的数据段如 `.data.rel.local` 或 `.data.rel`。同时，汇编中的函数调用准确体现了 x86-64 System V ABI 约定：前几个整型或指针参数优先通过寄存器传递，返回值通过 `%eax/%rax` 返回。该阶段说明，汇编程序并不是对 C 代码的逐字符翻译，而是把变量存储期、可见性和调用语义重组为节、符号和指令三个层面的机器级表示。

5.2.2 阶段 2：可重定位目标文件解析

1. 任务描述：本阶段要求生成 `main.o` 并结合 `readelf`、`objdump`、`hexdump` 等工具，分析节头表、数据节、符号表、重定位节和只读节中的具体内容，说明一个全局变量定义语句在可重定位目标文件中会留下哪些信息。

2. 实验设计：为了避免只看到零散命令输出而缺乏整体逻辑，本阶段采用“先节头表、再数据节、再重定位、最后符号表”的分析路径。其核心思想是：先用节头表建立文件的全局结构，再用 `.data/.bss/.rodata` 解释变量和常量的静态存储位置，再通过 `.rela.text`、`.rela.data.rel.local`、`.rela.data.rel` 等节

解释为什么某些地址在编译阶段还不能确定，最后借助 `.symtab` 与 `.strtab` 将“二进制中的字节布局”与“源程序中的符号含义”重新对应起来。

3. 实验过程：首先执行 `gcc -c -g -D U2 main.c -o main.o` 生成可重定位目标文件，然后用 `readelf -S -W main.o` 查看节头表，确认 `main.o` 中包含 `.text`、`.data`、`.bss`、`.rodata`、`.symtab`、`.strtab` 和多类 `.rela.*` 节。随后用 `readelf -x .data main.o` 验证 `init_gv1=10`、`init_gstr1="U202490042"`、`init_garr1=11,12,13` 和 `gv=6` 的实际字节排列；再用 `readelf -s main.o` 观察 `init_gv1`、`init_gstr1`、`init_garr1`、`gv` 等符号的 Value 与节内偏移之间的对应关系。之后继续查看 `.bss` 中的未初始化变量、`.data.rel.local` 和 `.data.rel` 中的带地址初值变量，以及 `.rela.text` 中对外部函数和全局地址访问形成的重定位条目。

4. 实验结果：实验结果显示，`main.o` 中共有 27 个主要节，其中 `.text` 负责存放代码，`.data` 负责存放已初始化普通全局变量，`.bss` 负责未初始化全局变量和静态局部变量，`.rodata` 负责只读常量，`.rela.*` 节则保存链接时需要修正的位置和符号信息。以 `.data` 节为例，十六进制转储中的 `0a000000`、`553230323439303034320000`、`0b000000` `0c000000` `0d000000` 与源程序里的 `init_gv1`、`init_gstr1`、`init_garr1` 一一对应。这说明一个全局变量定义语句并不会只在目标文件中留下“一个地方”的痕迹，而是会同时在数据节、符号表、字符串表甚至重定位节中产生可追踪的信息。

5.2.3 阶段 3：可执行目标文件解析

1. 任务描述：本阶段要求分别以 `main.c sub.c` 和 `sub.c main.c` 的顺序链接生成 `main_sub` 与 `sub_main`，比较它们在函数排列、全局变量地址、符号表地址和反汇编结果上的差异，并据此说明链接时符号地址的确定规律以及可执行文件相对于可重定位目标文件的本质变化。

2. 实验设计：本阶段采用“同源不同顺序”的对照法。由于两个可执行文件使用的源代码完全相同，唯一改变的是输入目标文件的链接顺序，因此观察到的地址差异可以直接归因于节合并顺序和链接器的符号布局策略。实验重点从三个角度展开：一是比较 `nm -n` 输出中函数和全局变量的排列顺序，说明链接器如何按输入文件顺序拼接同类节；二是比较 `readelf -s` 中可重定位目标文件的节内偏移与可执行文件中的最终虚拟地址；三是比较 `objdump -dr main.o` 与 `objdump`

-d main_sub 中同一段 main 函数代码，说明重定位前后的机器码差异。

3. 实验过程：实验先执行 `gcc -g -D U2 main.c sub.c -o main_sub` 和 `gcc -g -D U2 sub.c main.c -o sub_main` 生成两个顺序不同的可执行文件，再用 `nm -n` 按地址排序查看符号。结果显示，在 main_sub 中 fcount、fsub、main 先于 fadd、check_gcc_condition 排列，而在 sub_main 中顺序恰好相反；数据段中 init_gv1、init_gstr1、gv 的最终地址也随链接顺序而变化。随后通过 `readelf -s main.o` 与 `readelf -s main_sub` 对照同名符号，确认可重定位目标文件中的 Value 只是节内偏移，而可执行文件中的值已经是最终虚拟地址。最后比较 `objdump -dr main.o` 与 `objdump -d main_sub`，观察 call 和 lea 指令中原来的占位字段如何被实际相对位移替换。

4. 实验结果：结果表明，链接器会先合并多个目标文件中的同类节，再按输入目标文件顺序在节内依次拼接内容，因此符号最终地址并非只由源代码定义位置决定，也受链接输入顺序影响。对 main.o 而言，fcount、main、init_gv1 等符号的值只是相对于 .text 或 .data 的偏移；而在 main_sub 或 sub_main 中，它们已经变成最终映像中的真实地址。与此同时，main.o 里原本仍携带 R_X86_64_PC32、R_X86_64_PLT32 等重定位条目的位置，在可执行文件中已经被具体机器码修正完成。这说明生成可执行文件的本质，就是把多个目标文件的节、符号和待定地址整合进同一地址空间，并通过重定位把所有“尚未填完的半成品代码”落实为可执行映像。

5.2.4 阶段 4：静态链接与动态链接生成执行程序的比较

1. 任务描述：本阶段要求同时生成动态链接版 main_sub_dyn 和静态链接版 main_sub_static，从文件大小、运行时依赖、ELF 内部结构以及库函数调用路径等角度比较两种链接方式生成程序的差异。

2. 实验设计：阶段 4 的设计重点是从“程序里有没有包含库实现”和“运行时还需不需要系统继续补全地址”这两个根本问题出发。实验分别用 `ls -lh` 对比文件大小，用 `file` 和 `ldd` 判断程序对共享库的依赖情况，用 `readelf -d` 查看是否存在动态段，用 `objdump -d` 观察对 printf 的调用究竟是经过 @plt 还是直接进入库函数实现，从而把静态链接和动态链接的差异落实到可见证据上。

3. 实验过程：实验依次执行 `gcc -g -D U2 main.c sub.c -o main_sub_dyn` 与 `gcc -g -static -D U2 main.c sub.c -o main_sub_static`。随后通过 `ls -lh`

观察两者大小，得到动态链接版约为 20K，静态链接版约为 884K；再通过 `file` 和 `ldd` 验证前者是 `dynamically linked` 且依赖 `libc.so.6`，后者是 `statically linked` 且 `ldd` 提示 `not a dynamic executable`。接着查看 `readelf -d` 的结果，确认动态链接版包含 `NEEDED`、`PLTGOT`、`JMPREL` 等动态段信息，而静态链接版没有动态段。最后对 `fcount` 中的 `printf` 调用进行反汇编比较，发现动态链接版调用的是 `printf@plt`，静态链接版则直接调用程序内部并入的 `_IO_printf`。

4. 实验结果：实验表明，动态链接程序体积更小，因为它只保存用户程序本身的代码和共享库定位所需的元数据；静态链接程序体积显著更大，因为它把所需库函数实现直接打包进可执行文件内部。动态链接程序在运行时仍依赖共享库和动态加载器，因此 ELF 内部保留 `.dynsym`、`.dynstr`、`.rela.plt`、`.dynamic` 等结构；静态链接程序则不再需要这些动态解析辅助节。从调用路径上看，动态链接依赖 PLT/GOT 完成共享库函数定位，而静态链接直接调用已并入程序的库函数实现。这说明两种链接方式在文件大小、程序独立性、共享能力和后续更新维护方式上都具有本质区别。

5.2.5 阶段 5：动态链接库中的函数定位方法

1. 任务描述：本阶段要求以 `printf` 为例，说明动态链接库函数的地址不是在编译时就固定下来的，而是通过 未定义符号 $\rightarrow$ PLT 入口 $\rightarrow$ GOT 槽位 $\rightarrow$ 动态重定位 $\rightarrow$ 动态加载器解析这一链条在装载时或调用前被补全。

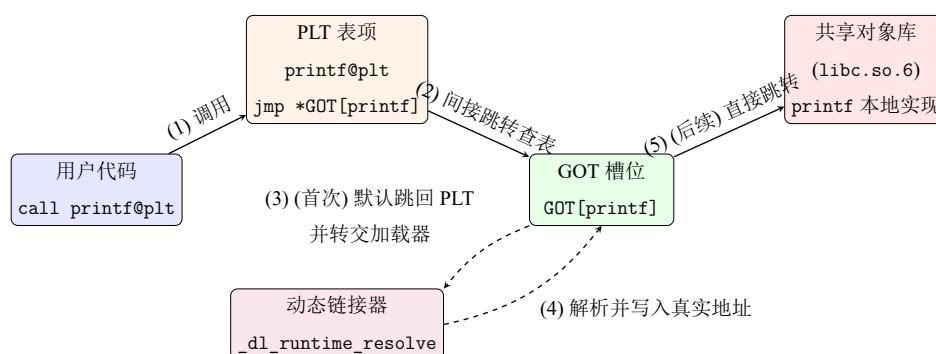


图 5-1 动态链接中借助 PLT 与 GOT 实现库函数延迟绑定与解析的过程示意图

2. 实验设计：为了避免把 PLT/GOT 讲成孤立概念，本阶段采用分层追踪法。首先在 `main.o` 中确认 `printf` 只是一个未定义外部符号；随后在动态链接可执行文件中找到用户代码对 `printf@plt` 的调用，再跟踪 `printf@plt` 如何通过 `jmp *GOT` 间接跳转；接着查看 `readelf -r` 和 `objdump -R` 中与 `printf` 对应的 `R_X86_64_JUMP_SLOT` 重定位项；最后用 `ldd` 和 `readelf -d` 说明动态加载器与

libc.so.6 在这个过程中分别扮演什么角色。

3. 实验过程: 实验先查看 `readelf -s main.o`, 确认 `printf` 的 `Ndx=UND`, 说明它在可重定位目标文件里只是一个未定义符号。然后转向 `main_sub_dyn`, 在 `objdump -d` 中观察到用户代码调用的是 `printf@plt` 而不是直接调用 `printf`。进一步反汇编 `printf@plt` 可以看到, 它通过 `jmp *...(%rip)` 跳向 GOT 表中的对应槽位。接着查看 `readelf -r main_sub_dyn` 与 `objdump -R main_sub_dyn`, 发现 `printf` 对应条目类型为 `R_X86_64_JUMP_SLOT`。最后结合 `ldd main_sub_dyn` 和 `readelf -d main_sub_dyn`, 确认该程序依赖 `/lib64/ld-linux-x86-64.so.2` 与 `libc.so.6`, 且当前实验环境中带有 `BIND_NOW` 标志, 因此符号通常在程序启动阶段就已完成解析。

4. 实验结果: 这一阶段表明, `printf` 的真实地址并不会直接写死在用户程序代码中, 而是先在可重定位目标文件中以未定义符号形式存在, 在动态链接可执行文件中获得一个 `printf@plt` 入口和一个 GOT 槽位, 再由动态加载器根据 `libc.so.6` 中的导出符号把真实地址写入 GOT。此后用户代码执行 `call printf@plt` 时, 就能间接跳转到动态库中的实际实现。也就是说, 动态链接库函数的定位不是“代码里直接存着最终地址”, 而是“代码、表项和重定位元数据共同配合动态加载器完成定位”。这也是动态链接能够在不把库实现直接拷入程序内部的前提下, 仍然正确调用外部库函数的根本原因。

5.3 实验小结

本次实验按照“汇编程序、可重定位目标文件、可执行文件、链接方式比较、动态函数定位”五个阶段逐层展开, 把同一份源程序在编译和链接流水线中的关键中间产物全部串联起来分析。阶段 1 让我看到变量和调用语义在汇编层如何落到伪指令、寄存器和寻址方式上; 阶段 2 说明一个全局变量定义语句实际上会在数据节、符号表、字符串表和重定位节中同时留下痕迹; 阶段 3 进一步说明链接器如何把多个目标文件的同类节合并、解析外部符号并完成重定位; 阶段 4 和阶段 5 则把分析范围从“本程序内部结构”扩展到“与共享库、动态加载器的交互机制”, 从而把静态链接、动态链接以及 PLT/GOT 机制联系起来理解。

从理论与方法上看, 本实验最重要的价值在于把 ELF 文件与链接理论转化成了一组可验证的文件结构事实。过去对 `.text`、`.data`、`.bss`、`.symtab`、`.rela.text` 等节的理解更多停留在概念层面, 而这次通过 `readelf`、`objdump` 和 `nm` 的逐项

观察，我能够直接把“节、符号、重定位、动态段、GOT、PLT”与具体的地址、字节和指令位置建立对应关系，也更清楚地理解了为什么链接顺序会改变符号地址、为什么未定义符号能够在链接后获得最终地址、以及为什么动态链接程序可以在体积更小的同时依赖共享库实现函数调用。

本次实验的主要收获有三点。第一，我对从源程序到可执行程序这一完整构建链路形成了更清晰的整体认识，理解了每个阶段产物各自承担的职责。第二，我掌握了通过 `readelf`、`objdump`、`nm`、`ldd` 互相印证的分析方法，而不是只依赖单个工具片面判断。第三，我对静态链接与动态链接、以及动态加载器如何借助 PLT/GOT 定位库函数有了更扎实的机器级理解。后续若继续完善本实验，还可以把相关截图进一步整理成“同一符号在不同阶段文件中的连续证据链”，并补充对 PIC、延迟绑定与立即绑定差异的更细致讨论，使分析更加系统完整。

6 实验总结

通过本学期《计算机系统基础》系列实验，我对计算机系统的认识比以前更全面、更具体了。过去在课堂上接触数据表示、汇编语言、机器级程序、缓冲区溢出和程序链接这些内容时，更多只是理解书上的概念，知道它们分别讲了什么，但并没有真正体会到这些知识之间是怎样联系起来的。经过这几次实验，我开始把课本中的理论和程序的实际运行过程对应起来，也对“一个程序是怎样一步步运行起来的”有了更清楚的认识。

从实验内容来看，每一次实验都让我有不同的收获。前面的实验让我进一步理解了数据在计算机中的存储方式，也让我认识到看似简单的按位运算和数据转换，其实都建立在严密的二进制规则之上。通过汇编和 C 语言混合编程实验，我明白了高级语言中的循环、判断、参数传递和函数调用，在底层最终都要落实到寄存器、指令和内存操作上。二进制炸弹实验则锻炼了我阅读汇编、分析程序逻辑的能力，让我学会从程序现象出发，逐步推断出内部的判断条件。

后面的两个实验让我对系统底层和程序安全有了更深的印象。缓冲区溢出实验让我直观地看到，如果程序缺少边界检查，就可能影响函数返回地址，甚至改变程序的执行流程。这让我认识到，安全问题并不是孤立存在的，而是和内存结构、函数调用、栈帧布局等基础知识紧密相关。ELF 文件与程序链接实验则让我了解了源程序从编译、链接到装载和运行的整个过程，也让我对目标文件、可执行文件、静态链接和动态链接之间的区别有了更清晰的理解。

通过这些实验，我最大的收获不仅是学会了一些工具和分析方法，更重要的是逐渐形成了先观察现象、再查找依据、最后结合理论进行判断的思路。当然，我也发现自己在阅读较复杂的汇编代码时还不够熟练，对一些细节问题还需要继续加强。今后我会继续整理实验中的关键知识，提升动手能力和分析能力，把课堂所学真正转化为理解系统和解决问题的能力。

参考文献

- [1] BRYANT R E, O'HALLARON D R. Computer Systems: A Programmer's Perspective[M]. 3rd ed. Boston : Pearson, 2016.
- [2] GNU Project. Debugging with GDB[EB/OL]. 2025 [2026-04-27].
<https://sourceware.org/gdb/current/onlinedocs/gdb/>.
- [3] 计算机系统基础实验指导书 [K]. 2026.